

AeroMind — Phase 3 Submission Packet

Agentic Systems Studio · Track A: Technical Build

Dhiksha Rathis · Sai Karthik · Smridhi Patwari · Tina Sibbal

April 2026

Phase 3 Submission Packet

Project title

AeroMind — The AI Operations Brain for Air Cargo

Team members

Name	Phase 3 ownership
Dhiksha Rathis	Agent architecture & orchestration, evaluation plan, governance tests
Sai Karthik	LoadIQ agent, tool registry & allowlist, FastAPI app, Docker Compose, evidence-capture scripts
Smridhi Patwari	ClearPath agent, prompt-injection filter, Next.js ops UI, interaction traces, README and final report
Tina Sibbal	CargoComply agent, LLM-as-judge, audit hash chain, PostgreSQL schema and governance views

Selected track

Track A — Technical Build.

One-paragraph project summary

AeroMind is a multi-agent AI platform for air-cargo hub operations controllers. A LangGraph-based hierarchical orchestrator dispatches three specialist agents — **LoadIQ** (load optimization), **ClearPath** (disruption rerouting), and **CargoComply** (regulatory compliance) — behind a seven-control governance

layer (DG lock, blast-radius cap, tool allowlist, prompt-injection sanitizer, SHA-256 audit hash chain, LLM-as-judge, and human-in-the-loop gating). The system routes real ops events (`NEW_BOOKING`, `WEATHER_ALERT`, `MANIFEST_CHANGE`, etc.) through coordinated agent fan-out, preserves state in Postgres + pgvector, and produces a tamper-evident audit trail. Phase 3 delivers a runnable end-to-end system and an evidence package scored against two complementary lenses: an in-house six-dimension coverage matrix (8/8 passing scenarios) and the CLASSic multi-dimensional framework via a pre-registered 630-trial ablation study that measures Accuracy, Security, Cost, Latency, and Stability for AeroMind against two baselines.

Repository link

GitHub: <https://github.com/drathis1/AeroMind> **Branch at submission:** `phase-3` **Evidence-capture commit:** `3922b685e28444ad9f019db446dfd5573b20947e` — the commit under which `pytest_phase3_run.txt`, all JSON traces in `traces/`, and the 630-trial CLASSic ablation were produced. **Submission HEAD:** `e7e7783748a9aaa75682a0378f4eb4bd52a88e1e` — docs + report-PDF export commits on top of the evidence capture; re-runs of `eval/capture_traces.py` and `eval/run_classic_experiment.py` reproduce identical outputs.

5-minute video link

VIDEO: <https://drive.google.com/file/d/1Q5czRLLhPpIQSR4hRwT4SjW2nHern6Dh/view>

Hosted on Google Drive (`Final Presentation Agentic AeroMind.mp4`). If the link prompts for sign-in, please request access or contact any team member.

Script for the recording lives at `../../media/demo_video_script.md` and is rubric-mapped for all eight required elements (problem, architecture, main workflow, coordination, evidence, failure, final artifact, product over slides).

Final report

The full final report is shipped as a separate PDF: `../../docs/final_report.pdf` (1.7 MB; source at `../../docs/final_report.md`). It covers all nine required sections:

1. Problem and user
2. Architecture and design choices
3. Implementation / build summary
4. Evaluation setup
5. Results (including the CLASSic 630-trial ablation)
6. Failure analysis
7. Governance, trust, and responsible behavior
8. Lessons learned and future improvements

9. Team contribution update

plus four appendices (file reference, reproduction commands, API surface, references).

Architecture diagram

- [../../../../docs/architecture_diagram.png](#) — five-layer system diagram (events → orchestrator → agents → governance → shared state + human gate)
- [../../../../docs/sequence_diagram.png](#) — WEATHER_ALERT two-phase fan-out sequence
- [../../../../docs/classic_radar.png](#) — CLASSic pentagon comparing AeroMind vs A1 (no-governance) vs A2 (flat sequential)

Mermaid source for the primary diagram: [../../../../docs/architecture_diagram.mmd](#) .

Screenshot index

Ten captioned PNGs in [../../../../docs/screenshots/](#) with index at [../../../../docs/screenshots/screenshot_index.md](#) :

File	What it shows
01_pytest_green.png	8/8 regression tests passing
02_trace_E2E01_happy_path.png	LoadIQ + CargoComply parallel fan-out, CLOSED_CLEAN
03_trace_E2E02_two_phase.png	Weather reroute — Phase 1 ClearPath → Phase 2 fan-out
04_failure_G0V01_dg_lock.png	DG lock zeroes autonomous commit
05_failure_G0V02_blast_radius.png	Blast-radius cap halts at cap+1
06_judge_JDG01_ungrounded.png	LLM-as-judge flags ungrounded compliance claim
07_injection_INJ01_redaction.png	Prompt-injection filter redacts external text
08_escalation_ESC01_human_gate.png	Human-gate escalation pauses workflow
09_audit_AUD02_tamper_detected.png	Hash-chain detects tampered audit row
10_api_live_demo_pipeline.png	Live demo API round-trip

Evaluation summary

- **In-house scenarios executed (Phase 3):** 8 / 8 PASS across the six dimensions (end-to-end, governance, escalation, adversarial, judge, audit). Files: [../../../../eval/test_cases.csv](#) , [../../../../eval/evaluation_results.csv](#) , [../../../../eval/pytest_phase3_run.txt](#) .

- **CLASSic ablation study:** 630 trials (7 scenarios × 3 architectures × 30 repetitions). Full: A0 (AeroMind, full governance) vs A1 (no-governance) vs A2 (flat sequential ReAct). Accuracy 1.00 / Security 1.00 with $\sigma = 0$ for A0; 0.40 / 0.43 for A1 and A2. Cohen's $d = 2.02$ and 1.63 . Files:
[../../eval/classic_methodology.md](#), [../../eval/run_classic_experiment.py](#),
[../../eval/classic_runs.csv](#), [../../eval/classic_summary.csv](#),
[../../eval/classic_pairwise.csv](#), [../../eval/ablations/](#), [../../eval/metrics/](#).
- **Failure cases (three documented):** FL-001 (DG-lock containment), FL-002 (blast-radius cap containment), FL-003 (evidence-path iteration — live full-mode API returned HTTP 500 without Postgres; pivoted to in-memory demo pipeline for screenshot 10, added demo-vs-full-mode README docs, captured 8 multi-mode responses under `outputs/sample_runs/`). Narrative:
[../../eval/failure_log.md](#), [../../eval/failure_analysis.md](#). Includes an honest limitation note on `CLOSED_CLEAN` vs `AWAITING_HUMAN` semantics (FL-001 next-step improvement).
- **Version notes:** [../../eval/version_notes.md](#) (commit, Python 3.13.7, pytest 9.0.2).

List of submitted files and folders

```
AeroMind-1/
├── README.md                # project overview + run instructions
├── AI_USAGE.md              # AI-tool disclosure
├── pyproject.toml           # Python package spec
├── docker-compose.yml       # Postgres + pgvector for full mode
├── aeromind/
│   ├── agents/              # orchestrator, agents, governance, API
│   ├── orchestrator/        # LoadIQ, ClearPath, CargoComply
│   ├── audit/               # LangGraph routing, state, graph
│   ├── injection/           # SHA-256 hash chain
│   ├── judge/               # prompt-injection filter
│   ├── tools/               # LLM-as-judge
│   ├── api/                 # tool registry + allowlist
│   ├── demo/                # FastAPI /v1 routes
│   └── db/                  # in-memory demo pipeline (/api/demo/*)
├── web/                     # Postgres schema + repositories
├── tests/                   # Next.js operator UI
├── eval/                    # unit tests (8/8 passing)
│   ├── test_cases.csv
│   ├── evaluation_results.csv
│   ├── failure_log.md
│   ├── failure_analysis.md
│   ├── version_notes.md
│   ├── pytest_phase3_run.txt
│   ├── capture_traces.py    # regenerates JSON traces
│   ├── render_screenshots.py # regenerates PNG evidence
│   ├── run_classic_experiment.py # 630-trial ablation driver
│   ├── classic_methodology.md # pre-registered protocol
│   ├── classic_runs.csv     # raw 630-row trial data
│   ├── classic_summary.csv  # per-arch × per-dim mean/σ
│   ├── classic_pairwise.csv # Cohen's d, p-values
│   ├── ablations/           # A0, A1, A2 runners
│   └── metrics/             # classic_score.py
├── traces/                  # 8 scenario JSON traces
├── docs/
│   ├── final_report.md
│   ├── final_report.pdf    # THE report (1.7 MB)
│   ├── architecture_diagram.mmd
│   ├── architecture_diagram.png
│   ├── sequence_diagram.png
│   ├── classic_radar.png   # Figure 3
│   ├── render_architecture.py
│   ├── render_sequence.py
│   ├── render_classic_radar.py
│   ├── _pandoc_header.tex
│   └── screenshots/        # 10 PNGs + screenshot_index.md
├── outputs/
│   └── sample_runs/        # 8 representative JSON outputs
├── media/
│   ├── demo_video_link.txt # paste URL here
│   └── demo_video_script.md # 5-min, 4-speaker, rubric-mapped
└── phase_submissions/phase3/
```

```
|— README.md
|— submission_checklist.md
|— submission_packet.md / .pdf # this document
|— ai_transcript_excerpts.md
|— reflections/                # dhiksha.md, sai.md, smridhi.md, tina.md
```

Known limitations (disclosed on the record)

1. The 8 Phase 3 in-house scenarios used **deterministic mocks** for external APIs and LLM calls. The CLASSic ablation is a controlled simulation. No live Gemini calls are embedded in the submitted evidence. The heuristic-only path of the judge is what's exercised in the test suite.
2. Scenario GOV-01 correctly contains the unsafe action (autonomous commit zeroed, booking never written) but ends with workflow `status = CLOSED_CLEAN` rather than the intended `AWAITING_HUMAN`. Fix queued; disclosed in `eval/failure_analysis.md`.
3. Phase 2 referenced 35 scenarios; Phase 3 executed 8. The remaining 27 are tracked as Phase 4 follow-ups in `docs/final_report.md §8`.
4. The full-mode API (`POST /v1/workflows/run`) requires Postgres. When `docker-compose up -d` is not running, the route returns HTTP 500 — exactly what FL-003 documents. For a reviewer who does not want to stand up the database, the in-memory `/api/demo/*` pipeline (screenshot 10 and `outputs/sample_runs/00-04`) exercises the same sequential three-agent flow with no external dependencies.

Assembled for Canvas submission. Render to PDF with:

```
pandoc phase_submissions/phase3/submission_packet.md \
-o phase_submissions/phase3/submission_packet.pdf \
--pdf-engine=xelatex \
-H docs/_pandoc_header.tex
```